

Assignment-3

22BCN1068

DATE: 16/3/29

PAGE NO: 21

Compiler Construction

Section - A (2x5=10)

1. What are the functions performed in synthesis phase.

Ans The synthesis phase of a compiler is responsible for generating the final output from the intermediate representation of the source code. It involves several key functions, including intermediate code generation, code optimization and code generation along with symbol table management and error handling to ensure efficient and error-free execution.

2. What are the goals of semantic analysis?

Ans The goal of semantic analysis is to ensure meaningful and error-free code by performing type checking, scope resolution, function binding and detecting semantic errors like undeclared variables and type mismatches.

3. What is syntax-directed definition?

Ans A syntax directed definition (SDD) is a formalism that associates semantic rules with grammar productions to specify the translation of language construct. It defines attributes (synthesized or inherited) and rules for computing them, helping in syntax-directed translation and semantic analysis.

4. What is syntax-directed translation?

Ans Syntax-Directed Translation (SDT) is a method of translating source code based on grammar rules, where semantic actions are embedded with the production rules. It is used to generate intermediate code, perform type checking, and enforce semantic rules during parsing.

5. Differentiate between S-attributed Definition and L-Attributed Definition.

Ans



S-Attributed Definition

- (i) Uses only synthesized attributes.
- (ii) Evaluated bottom-up (Postorder traversal).
- (iii) Suitable for LR parsing.

L-Attributed Definition

- Uses both synthesized and inherited attributes.
- Evaluated left-to-right (top-down).
- Suitable for LL parsing.

Section - B (7 x 3 = 21)

1. Differentiate b/w synthesized translation and inherited translation.

Ans Synthesized Translation

- (i) Uses synthesized attributes, which are computed from child nodes and passed up the parse tree.
- (ii) Evaluated in bottom-up parsing.
- (iii) ex:- Calculating the value of an expression in an expression tree.
- (iv) Used in S-Attributed grammars.
- (v) Simple to implement in LR parsers.
- (vi) Ensures information flows only upward.

Inherited translation

- Uses inherited attributes, which are passed down or across the parse tree from parent or sibling nodes.
- Evaluated in top-down parsing.
- ex:- Enforcing type compatibility b/w a variable and its assignment.
- Used in L-Attributed grammars.
- Works best with LL parsers.
- Allows information to flow in multiple directions.

2. What is Syntax Directed translation (SDT)?

Ans = Syntax Directed Translation (SDT) is a technique used in the compiler to generate intermediate code or target code based on the syntax of a program. It associates semantic rules with grammar productions, allowing translation during parsing. It has several key points -

- (i) Syntax-Directed Definition (SDD): - Defines how syntax influences translation, linking grammar productions with semantic

actions.

(ii) Attributes -

- Synthesized : Calculated from child nodes (bottom-up)
- Inherited - Passed from parent or siblings (top-down)

(iii) Semantic Rules - Specify how attributes are computed and guide translation.

(iv) Translation Scheme - Combines grammar rules with semantic actions.

(v) Role in Compiler - Help in syntax and semantic analysis, ensuring both syntactic correctness and meaningful translation.

Ex: For an expression $E \rightarrow E_1 + E_2$
The rule computes $E.val = E_1.val + E_2.val$

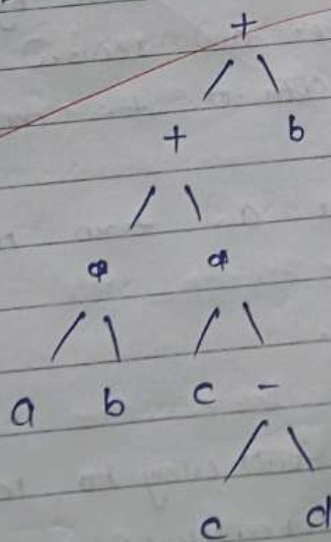
Q. Draw the syntax tree and DAG for the following expression:-
 $(a + b) + (c - d) * (a + b) + b$

Ans

(i) Syntax Tree - The syntax tree represents the hierarchical structure of the expression based on operator precedence.

Now, $(a + b) + (c - d) * (a + b) + b$

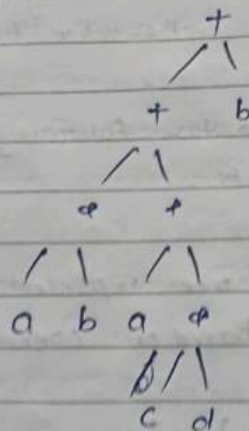
Syntax tree:-



- root is the '+'.
- left child = sum of $a+b$ and $(c-d) \times (a+b)$.
- The right child = b .
- The left child of the first '+' is the $a+b$.
- The right child of first '+' is the mul $(c-d) \times (a+b)$.

(ii) Directed Acyclic Graph (DAG): - A DAG is a graph that eliminates redundant computation by sharing common subexpressions. It reduces the expression into a more efficient representation by reusing subexpressions.

Now DAG:-



- The DAG eliminates the repeated $a+b$ by creating a shared node.
- $a+b$ is computed once and reused in both the left and right sides of the multiplication.
- ' $c-d$ ' is computed once, and then multiplied by $a+b$.
- root is the addition.

DAG provides optimized ~~graph~~ way to represent the expression by eliminating redundant subexpressions.

• Section - C

(1) Write short note on:

(a) Synthesized attributes :- Synthesized attributes are attributes whose values are computed from the attribute values of child nodes in the parse tree.

• These attributes are evaluated bottom-up during parsing.

ex:-

$$E \rightarrow E1 + T \quad \{ E.val = E1.val + T.val \}$$

(b) Inherited attributes :- Inherited attributes are attributes whose values are computed from the parent or sibling nodes in the parse tree.

• These attributes are evaluated top-down during parsing.

ex:-

$$S \rightarrow ABC \quad \{ B.inh = S.inh, C.inh = B.syn \}$$

(c) Dependency Graph :- A dependency graph represents the dependencies between attributes in a syntax directed translation (SDT).

• it used to determine the order of attribute evaluation in syntax - directed definitions.

ex: if $E.val = E1.val + T.val$ then $E1.val$ and $T.val$ must be evaluated before $E.val$.

(d) Evaluation order :- Evaluation order determines the sequence in which attributes in a syntax - directed definition are computed.

• Types:-

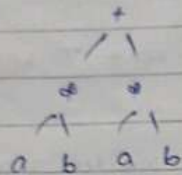
- (i) Top-down (Left-to-Right) - Used for inherited attributes.
- (ii) Bottom-up (Right-to-Left) - Used for synthesized attributes.

Goal :- Ensures no attribute is evaluated before its dependencies.

(e) Directed Acyclic Graph (DAG): - A DAG is a graph representation used in compiler optimization to eliminate common subexpressions and optimize computations.

ex: $E = (a * b) + (a * b)$

The DAG avoids redundant computations:



2. Construct SDT for given grammar:
 $E \rightarrow E + T, E \rightarrow T, T \rightarrow T * F, T \rightarrow F, F \rightarrow \text{digit}$
 Construct Annotated parse Tree for $5 * 4 + 3$.

Ans Step 1:

Given grammar with Syntax-Directed Translation Rules.
 - we define synthesized attributes to compute the expression's value.

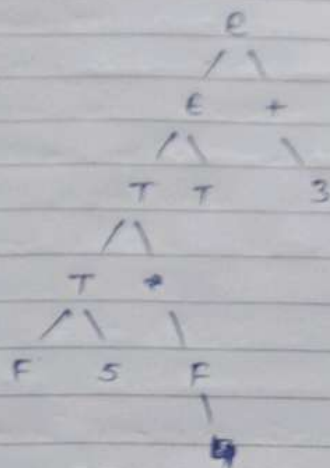
(i) Grammar with SDT Rules:-

1. $E \rightarrow E + T$ $\{ E.val = E.val + T.val \}$
2. $E \rightarrow T$ $\{ E.val = T.val \}$
3. $T \rightarrow T * F$ $\{ T.val = T1.val * F.val \}$
4. $T \rightarrow F$ $\{ T.val = F.val \}$
5. $F \rightarrow \text{digit}$ $\{ F.val = \text{digit.lexval} \}$

Step 2:

Parse Tree Construction for $5 * 4 + 3$

Now, we construct the parse tree based on the given grammar.



- Step-3: Annotating the parse tree (Attribute evaluation)
Compute value from the bottom up.

(i) Leaf Nodes ($F \rightarrow \text{digit}$)

ini -

$$F.val = 5$$

$$F.val = 4$$

$$F.val = 3$$

(ii) $T \rightarrow T * F$

ini -

$$T.val = 5 * 4 = 20$$

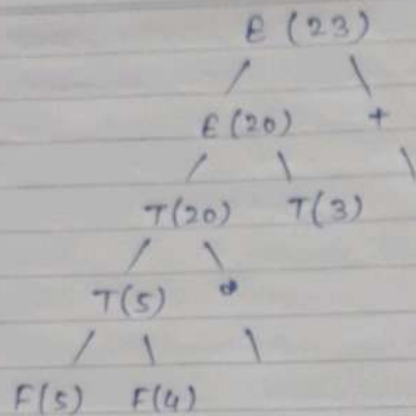
(iii) $E \rightarrow E + T$

ini -

$$E.val = 20 + 3 = 23$$

- Step 4: Final Annotated Parse Tree with values.





- Final Result: The value of $5+4+3 = 23$

3. Construct a DAG for the basic block whose code is given below.

1. $D := B + C$
2. $E := A + B$
3. $B := B + C$
4. $A := E - D$

Ans Step 1: Given code

1. $D := B + C$
2. $E := A + B$
3. $B := B + C$
4. $A := E - D$

Step 2: Identify Common Subexpressions

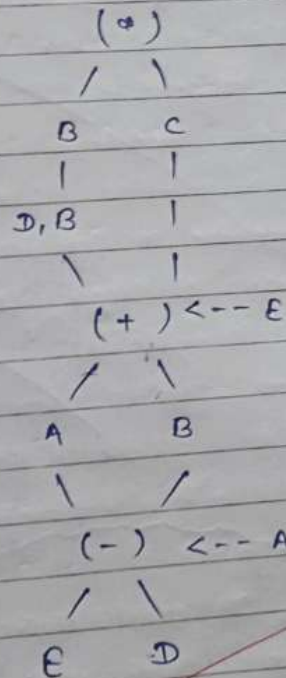
- $B + C$ appears twice in statements 1 & 3.
- Instead of computing $B + C$, we use the same result.
- E depends on $A + B$.
- $A := E - D$ is the final assignment.

Step 3: Constructing the DAG

Nodes in the DAG

- (i) Multiplication ($B \times C$)
 - Compute once and assign it to both D & B .
- (ii) Addition ($A + B$)
 - Compute and store in E .
- (iii) Subtraction ($E - D$)
 - Compute and assign to A .

Step 4: DAG Representation



Step 5: Explanation of the DAG Construction-

- (i) First Multiplication ($B \times C$) - The result is used in both D & B .
So we create a single Mul. node.
- (ii) Addition ($A + B$):
- The sum is stored in E .
- (iii) Subtraction ($E - D$)
- The result is assigned to A .

- * Step 6: Optimized code from DAA.
- Using the DAA, we optimize redundant computations.

1. $T1 := B + C$ (store result in 'T1')
2. $D := T1$
3. $B := T1$ (Reuse 'T1', avoiding recomputation)
4. $E := A + B$
5. $A := E - D$

This eliminates redundant computations of $B + C$.

